Predictive Analytics and Recommender Systems

Predictive Analytics - Machine Learning

Prakash B. Pimpale

June 07-09, 2018

1 Predictive Analytics - Machine Learning

1.1 Overview

- Introduction
- Regression
  - Linear Regression
- Classification
  - Logistic Regression
  - Naive Bayes
  - Decision Tree
  - SVM
- Model Improvement, Evaluation/Validation
  - Improving Performance of model
  - Confusion Matrix
  - Precision / Recall
  - Bias and Variance problems

1.2 Introduction

- Predictive analytics : Analysing current and historical facts to make predictions about future, or otherwise unknown, events
- Machine Learning methods are widely used for prediction in various domains - business, government, health care, etc.
- Machine Learning - Field of study that gives computers the ability to learn without being explicitly programmed
- Machine Learning based predictions - almost everywhere - stock market prediction, next key prediction, self driving cars, fraud detection, loan approvals, spam classification
- Involves
  - Training phase: Learn a model from training data
  - Testing phase : Test the model for accuracy
  - Prediction phase: Use the trained model to predict the unknown or future outcome. Field use.

(c) 2018 CDAC, Mumbai. All Rights Reserved

- An example:
  - Numeric data : 1 2 3 4 5 6 7 8 9 10
    - 1 2 3 4 5 | 6 7 8 9 10
    - 1 3 5 7 9 | 2 4 6 8 10
    - Classified into n <= 5 and others & even and odd classes
  - Text Data: A - 'click here to get free money and medicines', B - 'Dear Ram, Click here to see list of prescribed medicines by Dr. Ganesh. Also find attached reciept of money you paid. Regards, Disease Free Hospital, Mumbai.'
    - A: a spam | B: not spam

```
                                                        New Data
                                                           |
                                                           |
                                                           V
Training Data ===> Taining Algorithm ===> Model[Hypothesis]
                                                           |
                                                           |
                                                           V
                                                     Output/Prediction
```

- Training Data

House Size	Plot Size	Bedrooms	Granite	Bathroom	Selling Price
3529	9191	6	0	0	205000
3247	10061	5	1	1	224900
4032	10150	5	0	1	197900
2397	14156	4	1	0	189900
2200	9600	4	0	1	195000
3536	19994	6	1	1	325000
2983	9365	5	0	1	230000

- Different tasks, different models
  - Regression models - Numeric/continuous value prediction - Stock market price, Movie Ratings, Housing Prices

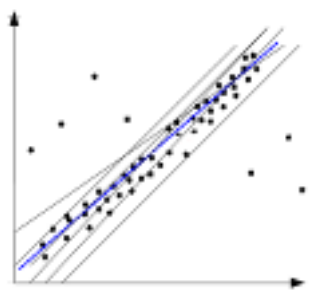
(c) 2018 CDAC, Mumbai. All Rights Reserved

  - Classification models - Categorical predictions/discrete values - defaulter prediction, document classification

- Suitability for our tasks depends on dimensions (no. of features), type (textual, numeric, etc) and size of the data
- Pre-Processing may be needed to convert it into suitable/better form - recoding, scaling, engineering
- These models can be evaluated using various metrics - Precision, Recall, F-measure etc.

1.3 Regression

- A machine learning technique that models relationship between a dependent (target) and independent variable(s) (predictor)
- Predicting housing price based on area of an apartment, plot size, number of bedrooms, etc.
- Comes up with function of type Y = f(X) : Y - Dependent variable and X- Independent Variable and f() the regression function
- Multiple Regression lines (Axis labels X-Y) :



- Simple Linear Regression - single predictor variable - input variables - Y = mX+c, m: slope of line and c: Y-intercept
- Multiple Linear Regression (widely known as linear regression) - multiple predictor variables

1.3.1 Linear Regression

- Objective of linear regression is to fit a line to the training data and then predict values based on that
- The hypothesis generated by a linear regression :

$$h_\theta(X) = \theta_0 + \theta_1 X$$

- Where θ_0 and θ_1 are called parameters - to be obtained through training
- The training problem can be formulated as:

$$minimize_{\theta_0,\theta_1} J(\theta_0, \theta_1) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^i) - y^i)^2$$

(c) 2018 CDAC, Mumbai. All Rights Reserved

- $J(\theta_0, \theta_1)$ is the cost function - a cost because of the error (difference between actual and predicted values)
- An algorithm to get the $\theta_0 and \theta_1$ such that the cost $J(\theta_0, \theta_1)$ is minimum is called Gradient Decent

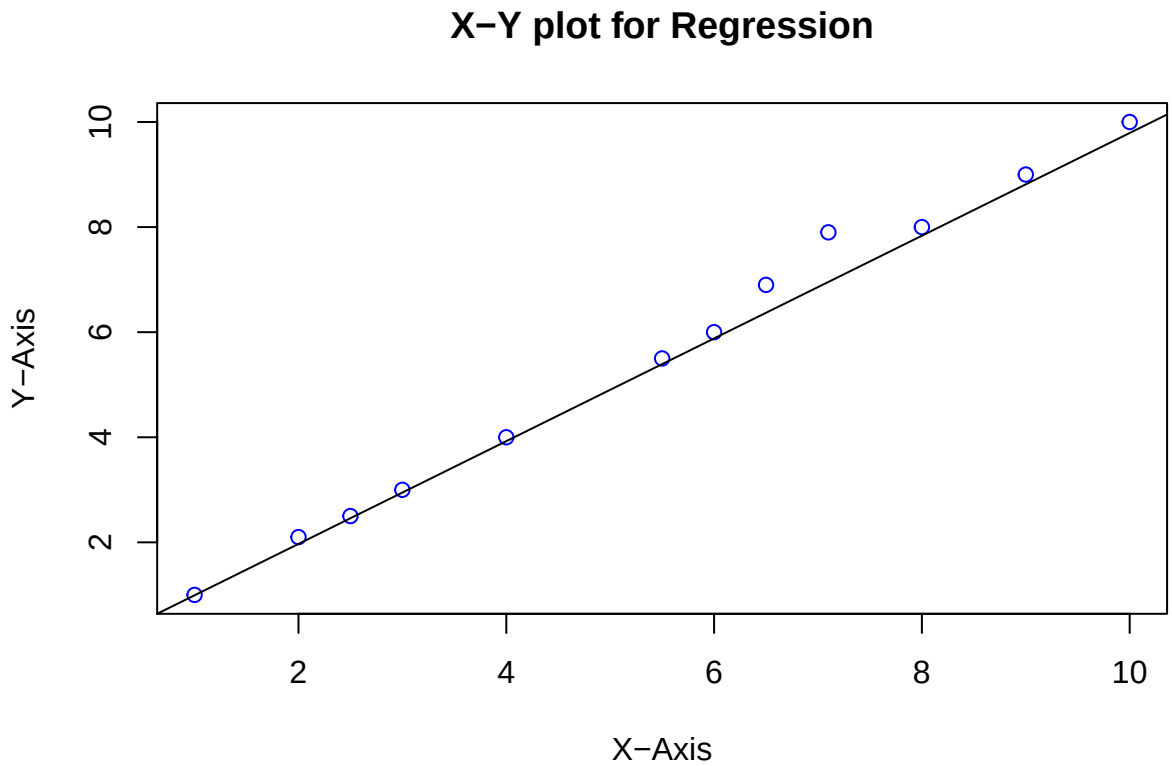
  repeate until convergence {

  $$\theta_j := \theta_j - \alpha \frac{d}{d\theta_j} J(\theta_0, \theta_1)$$

  }
- α is learning rate and decides how long steps the gradient decent should take
- The cost function for linear regression is convex function : bowl 'U' shaped
- Gradient decent updates parameter θ values such that the cost is minimum i.e. base of the bowl
- The derivative (partial) in the gradient descent is slope of the tangent to cost function at particular θ
- The slope decides the direction of change of θ, eventualy giving us the optimum parameters
- For plot with θ_0=0 and θ_1 approx 1

```
X <- c(1, 2, 2.5,3,4,5.5,6,6.5,7.1,8,9,10)
Y <- c(1, 2.1,2.5,3,4,5.5,6,6.9,7.9,8,9,10)
plot(X, Y, col = "blue", main = "X-Y plot for Regression", xlab = "X-Axis", ylab = "Y-Axis")
abline(lm(X ~ Y))
```


(c) 2018 CDAC, Mumbai. All Rights Reserved



Hypothesis is $h_\theta(X) = \theta_0 + \theta_1 X$ which is value of Y

$\theta = [\begin{smallmatrix} 0 \\ 1 \end{smallmatrix}]$ and so hypothesis becomes $h_\theta(X) = \theta_1 X$

Then, for X=4.5; $Y = h_\theta(X) = \theta_1 X = [1].[4.5] = 4.5$

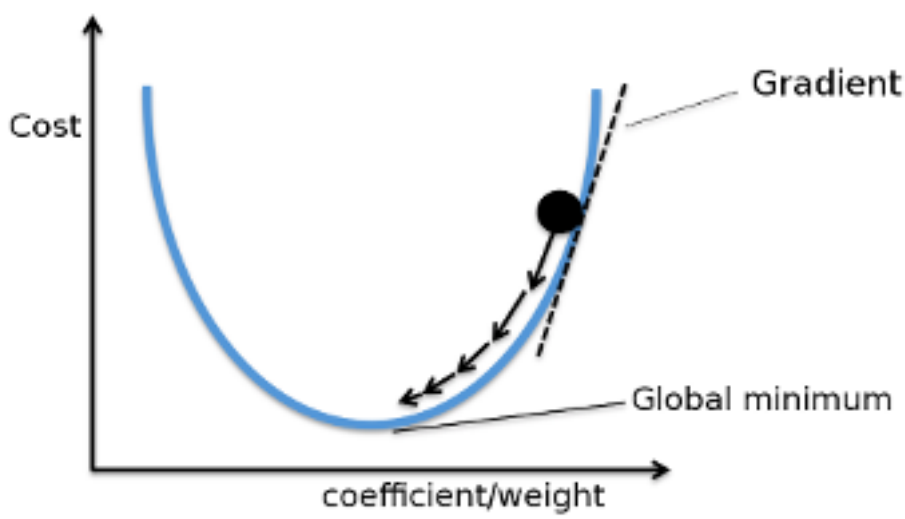


Figure 1: The cost function against coefficient θ_1

- WEKA [link] (http://www.cs.waikato.ac.nz/ml/weka/) A collection of Machine Learning

(c) 2018 CDAC, Mumbai. All Rights Reserved

algorithms for data analysis/mining tasks - Java Based

- An Example of Machine Learning/Simple Linear Regression

```
#Machine learns the model
# create a vector with numbers from 1 to 100
a <- 1:100
# create another vector with numbers a+2 of the same size as a
b <- a+2
# Create a DataFrame using these two vectors
data1 <- data.frame("Input"=a, "Output"=b)
# Randomize rows before training using sample function
data1 = data1[sample(nrow(data1), ), ]
#data1
# TRAIN - training creates a model
library(RWeka)
modelLR <- LinearRegression(Output ~ Input, data = data1)
# new variables for prediction
newdata1 <- data.frame("Input"=c(101:115))
newdata1
# Predict for new data
predictions <- predict(modelLR, newdata = newdata1)
predictions
# We can see how magically the relationship between numbers
# has been learned - validate it with what we established
modelLR
```

- Linear Regression with housing data

```
# load library
library(RWeka)
setwd("/home/prakash/Dropbox/myFolder/Link to resourcePARS/slides")
# read data from a file
data1 <- read.csv("code/houses.csv", header = T)
# See the data
data1
```

```
##   houseSize plotSize bedrooms granite bathroom sellingPrice
## 1      3529     9191        6       0        0       205000
## 2      3247    10061        5       1        1       224900
## 3      4032    10150        5       0        1       197900
## 4      2397    14156        4       1        0       189900
## 5      2200     9600        4       0        1       195000
## 6      3536    19994        6       1        1       325000
## 7      2983     9365        5       0        1       230000
```


(c) 2018 CDAC, Mumbai. All Rights Reserved

```r
# training
modelLR <- LinearRegression(data1$sellingPrice ~ ., data = data1)
# You can see the model and contribution of factors to the prediction
modelLR
```

```
##
## Linear Regression Model
##
## data1$sellingPrice =
##
##     -26.6882 * houseSize +
##       7.0551 * plotSize +
##   43166.0767 * bedrooms +
##   42292.0901 * bathroom +
##  -21661.1208
```

```r
# Create a new dataset for prediction - predicting for the 1st entry in training data
# you can take new
newdata1 <- data.frame(data1[1,1:5])
newdata1
```

```
##   houseSize plotSize bedrooms granite bathroom
## 1      3529     9191        6       0        0
```

```r
#prediction uisng the learned model
predictions <- predict(modelLR, newdata = newdata1)
# Is it approximately right?
predictions
```

```
## [1] 207996.2
```

1.4 Classification

- The outcome or prediction is a discrete value 0 or 1 or a categorical variable Male/Female - Binary Classification
- It can also be 0, 1, 2 or Politics, Entertainment, Education, Business - Multiclass classification

1.4.1 Evaluation of Model - Confusion Matrix

- Confusion matrix contains information about actual and predicted classes

(c) 2018 CDAC, Mumbai. All Rights Reserved

```
Classes : A and B : 10 entries each.
A - Postive class
B - Negative class

                                Predicted
                            A               B
        Actual          A   7               3
                        B   1               9
```

- True positives (TP): These are cases in which we predicted A, and they are actually A.
- True negatives (TN): We predicted B, and they are actually B.
- False positives (FP): We predicted A, but they are actually B (Also known as a "Type I error")
- False negatives (FN): We predicted B, but they are actually A (Also known as a "Type II error")

1.4.2 Precision / Recall

- Precision = proportion of the predicted positive cases that were correct
- Recall = proportion of positive cases that were correctly identified
- F measure = 2(Precision*Recall)/ Precision+Recall

```
Precision = TP/(TP+FP)    = (7/7+1) = 0.875
Recall =    TP/(TP+FN)    = (7/7+3) = 0.70
F1 Measure = 2(Precision*Recall)/ Precision+Recall = 2*0.875*0.70/(0.875+0.70) = 0.777
```

1.4.3 Logistic Regression

- A widely used classification method
- Basically decides value based on a threashold - on which side the data falls
- Logistic regression's hypothesis function : sigmoid/logistic - outputs values between 0 and 1 only

For linear Regrssion -

$$h_\theta(X) = \theta^T X$$

For logistic Regression -

$$h_\theta(X) = \frac{1}{1 + e^{-\theta^T X}}$$

- Can be trained using Gradient Decent

(c) 2018 CDAC, Mumbai. All Rights Reserved

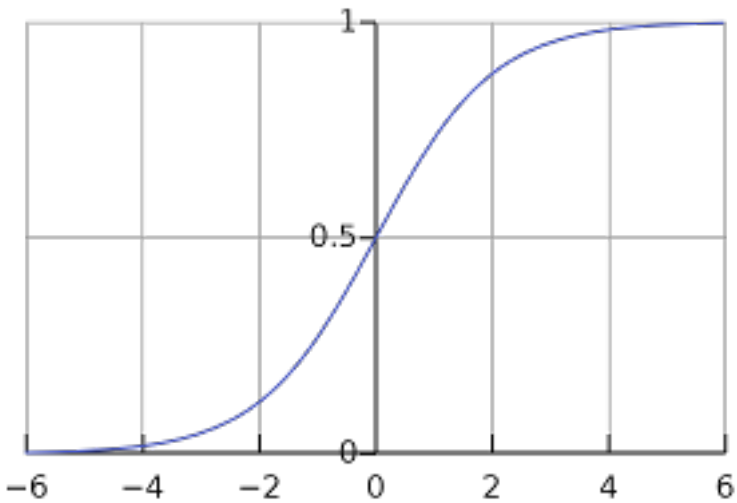


Figure 2: Logistic Function

```
# Logistic for defaulters
library(RWeka)
data1 <- read.csv("code/CreditDefaulters.csv", header = T)
# dimensions
dim(data1)
```

```
## [1] 1000   21
```

```
# what are headers
names(data1)
```

```
##  [1] "checking_balance"     "months_loan_duration" "credit_history"
##  [4] "purpose"              "amount"               "savings_balance"
##  [7] "employment_length"    "installment_rate"     "personal_status"
## [10] "other_debtors"        "residence_history"    "property"
## [13] "age"                  "installment_plan"     "housing"
## [16] "existing_credits"     "job"                  "dependents"
## [19] "telephone"            "foreign_worker"       "default"
```

```
# few samples
data1[1:3,]
```

```
##   checking_balance months_loan_duration credit_history   purpose amount
## 1           < 0 DM                     6       critical  radio/tv   1169
## 2       1 - 200 DM                    48         repaid  radio/tv   5951
```


(c) 2018 CDAC, Mumbai. All Rights Reserved

```
## 3          unknown                    12       critical education   2096
##   savings_balance employment_length installment_rate personal_status
## 1         unknown           > 7 yrs                4     single male
## 2        < 100 DM         1 - 4 yrs                2          female
## 3        < 100 DM         4 - 7 yrs                2     single male
##   other_debtors residence_history    property age installment_plan housing
## 1          none                 4 real estate  67             none     own
## 2          none                 2 real estate  22             none     own
## 3          none                 3 real estate  49             none     own
##   existing_credits                job dependents telephone foreign_worker
## 1                2   skilled employee          1       yes            yes
## 2                1   skilled employee          1      none            yes
## 3                1 unskilled resident          2      none            yes
##   default
## 1       1
## 2       2
## 3       1
```

```r
# Default status 1 means - is a defaulter and 2 means not.
# check the type of the class vector
class(data1$default)
```

```
## [1] "integer"
```

```r
# logistic in WEKA can't handle numeric classes and so the recoding of
# variable is needed as a factor - recode
data1$default <- as.factor(data1$default)
class(data1$default)
```

```
## [1] "factor"
```

```r
# train on first 900 samples and test on rest 100
trainingData <- data1[1:900,]
# test on the remaining samples
testiningData <- data1[901:1000,]
# train
modelLogiR <- Logistic(trainingData$default ~ ., data = trainingData)
#prediction on the testing data
predictions <- predict(modelLogiR, newdata = testiningData)
predictions
```

```
##   [1] 1 1 1 1 1 1 1 2 1 1 1 1 1 1 2 2 1 2 1 2 1 1 2 1 2 2 2 2 1 2 1 1 1 1 2
##  [36] 2 1 1 2 1 1 1 1 1 1 2 2 1 1 1 1 1 1 2 2 1 1 1 2 1 1 2 1 1 2 1 1 1 1 1
##  [71] 1 1 2 2 1 1 1 1 1 2 1 1 1 2 1 2 2 1 1 1 1 1 1 2 1 1 2 1 2 1
## Levels: 1 2
```


(c) 2018 CDAC, Mumbai. All Rights Reserved

```r
# get the confusion matrix - actual and predicted
table(testiningData$default,predictions)
```

```
##    predictions
##      1  2
##   1 56 12
##   2 14 18
```

```r
#Accuracy claculations
cm = as.matrix(table(testiningData$default,predictions))
#Total number of elements
n = sum(cm)
n
```

```
## [1] 100
```

```r
#diagonal elements - correctly classified
diag = diag(cm)
#Accuracy
accuracy = sum(diag) / n
#Print accuracy
accuracy
```

```
## [1] 0.74
```

1.4.4 Naive Bayes

- Based on the Bayes theorem
- Predicts based on probabilities from training data

```
P(B|A) = P(A|B) P(B)/P(A)

Gives posterior probability of 'B' given 'A'
        using
prior probability of 'B'
prior probability of 'A'
and conditional probability of 'A' given 'B'

B <-  Karina/Kajol? - The Class
A <- Sharukh

(What if A - Reema Lagoo, a famous mother in most
movies - A common appearance in data)
```


(c) 2018 CDAC, Mumbai. All Rights Reserved

- Takes two step approach
  - Calculates the posterior probability of the Class given the input - for every class
  - Assigns the class with higher posterior probability
- More suited when dimensionality of input is high the - widely used for document classification
- Also good for the multiclass classifications
- Works well with less datasets also, but the assumption that predictor variables are independent should hold

```
# loading library e1071
library(e1071)
# iris dataset
data(iris)
# training a naive Bayes model
model <- naiveBayes(Species ~ ., data = iris)
# Randomising data to get the all types of flowers
pData <- iris[sample(nrow(iris),),]
#Take samples from 11 to 20
pData <- pData[11:20,]
pData
```

```
##     Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
## 39           4.4         3.0          1.3         0.2     setosa
## 91           5.5         2.6          4.4         1.2 versicolor
## 4            4.6         3.1          1.5         0.2     setosa
## 108          7.3         2.9          6.3         1.8  virginica
## 7            4.6         3.4          1.4         0.3     setosa
## 100          5.7         2.8          4.1         1.3 versicolor
## 144          6.8         3.2          5.9         2.3  virginica
## 5            5.0         3.6          1.4         0.2     setosa
## 74           6.1         2.8          4.7         1.2 versicolor
## 71           5.9         3.2          4.8         1.8 versicolor
```

```
#Predict on these
preds <- predict(model, pData)
# Performance
table(pData$Species, preds)
```

```
##             preds
##              setosa versicolor virginica
##   setosa          4          0         0
##   versicolor      0          3         1
##   virginica       0          0         2
```


(c) 2018 CDAC, Mumbai. All Rights Reserved

1.4.5 Decision Tree

- Tree based method for classification
- Has root nodes, intermediate nodes and leaf nodes

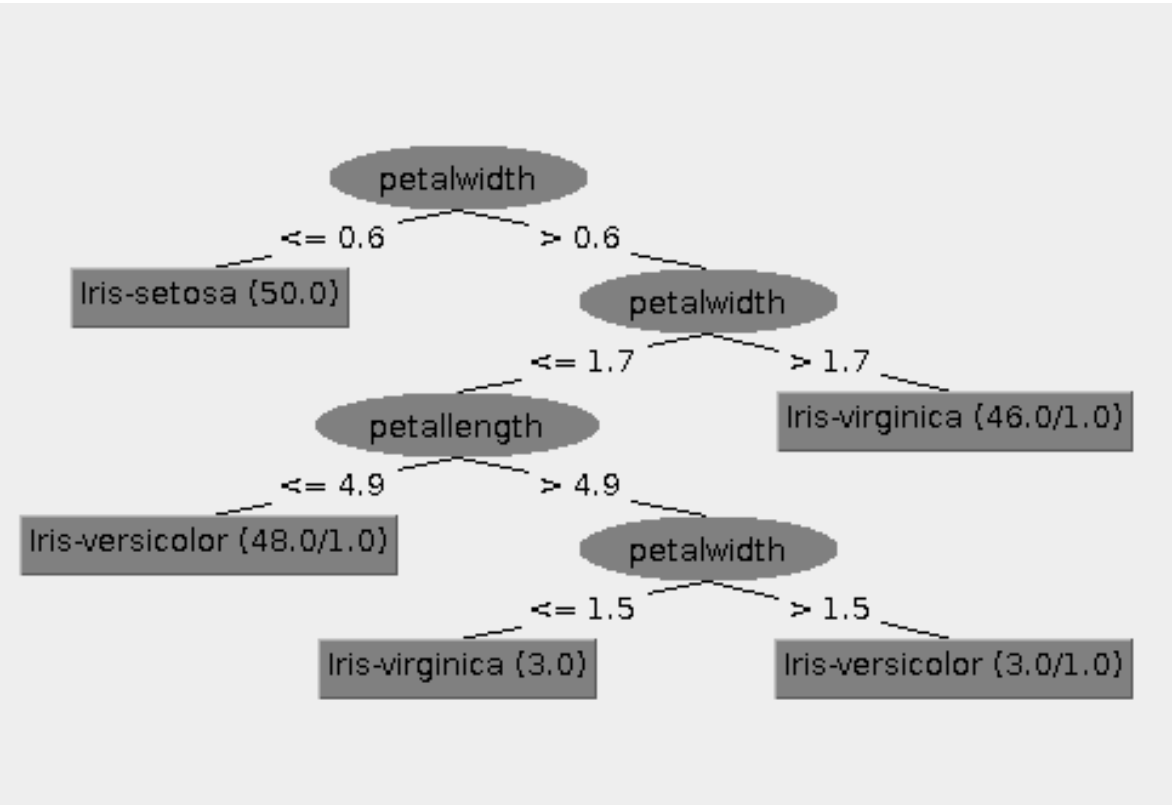


Figure 3:

- Reasonably fast in training
- Easy to understand and explainable model
- Can be converted to if-then rules
- Will have issues with increased sparseness - absent features

```
library(RWeka)
#summary(iris)
# trainig with complete data
iris_j48 <- J48(Species ~ ., data = iris)
# See the tree
iris_j48
```

```
## J48 pruned tree
```


(c) 2018 CDAC, Mumbai. All Rights Reserved

```
## ------------------
##
## Petal.Width <= 0.6: setosa (50.0)
## Petal.Width > 0.6
## |   Petal.Width <= 1.7
## |   |   Petal.Length <= 4.9: versicolor (48.0/1.0)
## |   |   Petal.Length > 4.9
## |   |   |   Petal.Width <= 1.5: virginica (3.0)
## |   |   |   Petal.Width > 1.5: versicolor (3.0/1.0)
## |   Petal.Width > 1.7: virginica (46.0/1.0)
##
## Number of Leaves  :  5
##
## Size of the tree :   9
```

```r
preds <- predict(iris_j48, pData)
# Performance
table(pData$Species, preds)
```

```
##             preds
##              setosa versicolor virginica
##   setosa          4          0         0
##   versicolor      0          3         1
##   virginica       0          0         2
```

```r
# see options
#WOW(J48)
```

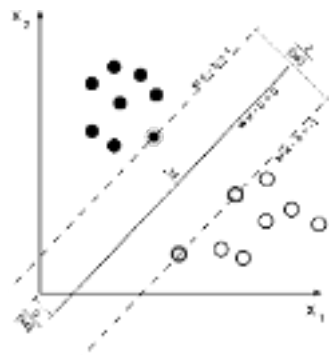
1.4.6 Random Forest

```r
library(randomForest)
# trainig with complete data
iris_rf <- randomForest(Species  ~ ., data = iris)
preds <- predict(iris_rf, pData)
# Performance
table(pData$Species, preds)
```

```
##             preds
##              setosa versicolor virginica
##   setosa          4          0         0
##   versicolor      0          4         0
##   virginica       0          0         2
```


(c) 2018 CDAC, Mumbai. All Rights Reserved

1.4.7 SVM



- A powerful classification algorithm - supervised
- Maximum margin classifier
- Works well for high dimensional data
- Basically takes your non-linearly separable data to linearly separable dimensions

X1	X2	O/P		X1	X2	X1.X2	O/P
1	1	0		1	1	1	0
-1	-1	0		-1	-1	1	0
1	-1	1		1	-1	-1	1
-1	1	1		-1	1	-1	1

Figure 4:

- May also work in cases where the number of dimensions are higher than the no. of samples
- Memory efficient as only the support vectors are used to decide class of the new candidate
- Good generalization - good perfomance on new data
- Found to be not woking well with very huge datasets - high training time
- Also doesn't work well if the target classes are overlapping

```
# Credit defaulters
data1 <- read.csv("code/CreditDefaulters.csv", header = T)
dim(data1)
```

```
## [1] 1000   21
```


(c) 2018 CDAC, Mumbai. All Rights Reserved

```r
# Recoding as factors
data1$default <- as.factor(data1$default)
#summary(data1)
# train on first 900 samples and test on rest 100
trainingData <- data1[1:900,]
# test on the remaining samples
testiningData <- data1[901:1000,]
# train
modelSVM <- SMO(trainingData$default ~ ., data = trainingData)
#prediction on the testing data
predictions <- predict(modelSVM, newdata = testiningData)
predictions
```

```
##   [1] 1 1 1 1 1 1 1 2 1 1 1 1 1 1 2 2 1 2 2 1 1 1 2 1 2 2 2 2 1 2 1 1 1 1 2
##  [36] 2 1 1 2 1 1 1 1 1 1 2 2 1 1 1 1 1 1 2 2 1 1 1 2 1 1 2 1 1 1 1 1 1 1 1
##  [71] 1 1 2 2 1 1 1 1 1 2 1 1 1 1 1 2 2 1 2 1 1 1 1 2 1 1 2 1 2 1
## Levels: 1 2
```

```r
# get the confusion matrix - actual and predicted
table(testiningData$default,predictions)
```

```
##    predictions
##      1  2
##   1 56 12
##   2 15 17
```

- An example with Caret

```r
# A widely used package for machine learning in R
library(caret)
#example
## Classification Example
data(iris)
TrainData <- iris[,1:4]
#TrainData
TrainClasses <- iris[,5]
#TrainClasses
MLModel <- train(TrainData, TrainClasses, method = "nnet")
#MLModel <- train(TrainData, TrainClasses, method = "knn")
#MLModel <- train(TrainData, TrainClasses, method = "bstTree")
#Now you may want to see other algorithsm supported by caret
#names(getModelInfo())
#Description of the package in details
```


(c) 2018 CDAC, Mumbai. All Rights Reserved

```
#https://www.jstatsoft.org/index.php/jss/article/view/v028i05/v28i05.pdf
predictions <- predict(object = MLModel, newdata = TrainData)
table(predictions,TrainClasses)
```

1.5 Model Improvement, Validation and other issues

1.5.1 Improving Performance of model

- Feature Selection - Select the best - corelation, chi2, mutual information
- Feature Engineering - engineer new features - position of a words in a sentence, POS tags of the words, sum the family salaries, children -> fees
- Handle missing values and outliers
- Scale data to proper ranges
- Tune the parameters of machine learning algorithm
- Increase data

1.5.2 Evaluation For Numeric Predictions

- p :predicted value and a: actual value

Courtesy: Data Mining: Practical Tools and Techniques (The WEKA book)

```
# Evaluating a regression model with insurance data
library(RWeka)
data1 <- read.csv("code/insuranceCharges.csv", header = T)
# train
modelLR <- LinearRegression(data1$charges ~ ., data = data1)
# evaluates on the training instances only unless new data is provided
eval_LR <- evaluate_Weka_classifier(modelLR, numFolds = 10, complexity = FALSE, seed = 1, cl
eval_LR
```

```
## === 10 Fold Cross Validation ===
##
## === Summary ===
##
## Correlation coefficient                  0.864
## Mean absolute error                   4211.6025
## Root mean squared error               6095.5645
## Relative absolute error                 46.2753 %
## Root relative squared error             50.2864 %
## Total Number of Instances             1338
```


(c) 2018 CDAC, Mumbai. All Rights Reserved

Mean-squared error	$\frac{(p_1 - a_1)^2 + \ldots + (p_n - a_n)^2}{n}$
Root mean-squared error	$\sqrt{\frac{(p_1 - a_1)^2 + \ldots + (p_n - a_n)^2}{n}}$
Mean-absolute error	$\frac{\lvert p_1 - a_1\rvert + \ldots + \lvert p_n - a_n\rvert}{n}$
Relative-squared error*	$\frac{(p_1 - a_1)^2 + \ldots + (p_n - a_n)^2}{(a_1 - \bar{a})^2 + \ldots + (a_n - \bar{a})^2}$
Root relative-squared error*	$\sqrt{\frac{(p_1 - a_1)^2 + \ldots + (p_n - a_n)^2}{(a_1 - \bar{a})^2 + \ldots + (a_n - \bar{a})^2}}$
Relative-absolute error*	$\frac{\lvert p_1 - a_1\rvert + \ldots + \lvert p_n - a_n\rvert}{\lvert a_1 - \bar{a}\rvert + \ldots + \lvert a_n - \bar{a}\rvert}$
Correlation coefficient**	$\frac{S_{PA}}{\sqrt{S_P S_A}}$, where $S_{PA} = \frac{\sum_i (p_i - \bar{p})(a_i - \bar{a})}{n-1}$, $S_P = \frac{\sum_i (p_i - \bar{p})^2}{n-1}$, $S_A = \frac{\sum_i (a_i - \bar{a})^2}{n-1}$

*Here, $\bar{a}$ is the mean value over the training data.
**Here, $\bar{a}$ is the mean value over the test data.

Figure 5:

(c) 2018 CDAC, Mumbai. All Rights Reserved

1.5.3 Cross-Validation

- Model validation technique
- Tells us about how good the model will generalize
- Generalisation V/S Overfitting
- 10 fold cross validation - split data into 10 sets - train on 9 and test on 1
- Using inbuilt evaluations

```
library(RWeka)
#summary(iris)
# trainig with complete data
iris_j48 <- J48(Species ~ ., data = iris)
# Se the evaulations or performance on training data
summary(iris_j48)
```

```
##
## === Summary ===
##
## Correctly Classified Instances         147               98      %
## Incorrectly Classified Instances         3                2      %
## Kappa statistic                          0.97
## Mean absolute error                      0.0233
## Root mean squared error                  0.108
## Relative absolute error                  5.2482 %
## Root relative squared error             22.9089 %
## Total Number of Instances              150
##
## === Confusion Matrix ===
##
##   a  b  c   <-- classified as
##  50  0  0 |  a = setosa
##   0 49  1 |  b = versicolor
##   0  2 48 |  c = virginica
```

```
eval_j48 <- evaluate_Weka_classifier(iris_j48, numFolds = 10, complexity = FALSE, seed = 1,
eval_j48
```

```
## === 10 Fold Cross Validation ===
##
## === Summary ===
##
## Correctly Classified Instances         144               96      %
## Incorrectly Classified Instances         6                4      %
## Kappa statistic                          0.94
```


(c) 2018 CDAC, Mumbai. All Rights Reserved

```
## Mean absolute error                       0.035
## Root mean squared error                   0.1586
## Relative absolute error                   7.8705 %
## Root relative squared error              33.6353 %
## Total Number of Instances               150
##
## === Detailed Accuracy By Class ===
##
##                  TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC
##                  0.980    0.000    1.000      0.980    0.990      0.985    0.990     0.98
##                  0.940    0.030    0.940      0.940    0.940      0.910    0.952     0.88
##                  0.960    0.030    0.941      0.960    0.950      0.925    0.961     0.90
## Weighted Avg.    0.960    0.020    0.960      0.960    0.960      0.940    0.968     0.92
##
## === Confusion Matrix ===
##
##   a  b  c   <-- classified as
##  49  1  0 |  a = setosa
##   0 47  3 |  b = versicolor
##   0  2 48 |  c = virginica
```

1.5.4 Bias and Variance problems

- Bias Problem:
  - The model is underfitting - not able to fit even the training data
  - High error rate on training data and High error rate on unseen (development set) data - Less difference
  - Lower order polynomials in the data - trying to use simple curves, say lines to seperate a non-linearly seperable data - less features
  - If you have Bias Problem increase your features
  - Add more polynominal features - ex. squares of the same features
  - Use Bagging (Bootstrap Aggregation) - parallel ensemble - train parallaly different models using different samples of training data - final o/p is weighted average of those all classifiers or voting based decision - Random forest (tree)
- Variance Problem:
  - The model is overrfitting - 'perfectly' fits training data - but fails on unseen data
  - Low error rate on training data and High error rate on unseen (development set) data - High difference
  - Higher order polynomials in the data - use of complex curves to seperate the data - more features
  - If you have variance problem decrease your features
  - Remove polynominal features - ex. squares/cubes of the same features
  - Use Boosting - Sequential Ensemble - train new models with more emphasis on data which previous one mis-classfied - Adaboost

(c) 2018 CDAC, Mumbai. All Rights Reserved

1.6 References

- Data Mining Practical Machine Learning Tools and Techniques Third Edition by Ian H. Witten, Eibe Frank, Mark A. Hall
- Machine Learning Courses by Andrew Ng on coursera (2012)
- Predictive Analytics using R By Jeffrey S. Strickland
- Machine Learning with R https://cran.r-project.org/web/views/MachineLearning.html

(c) 2018 CDAC, Mumbai. All Rights Reserved